

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Recap 3
Kalman Filter + Extended + Unscented
Multi-Object Tracking

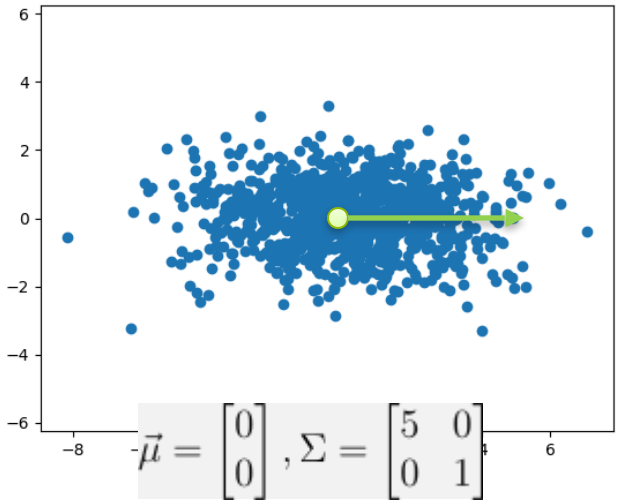
June 24, 2026

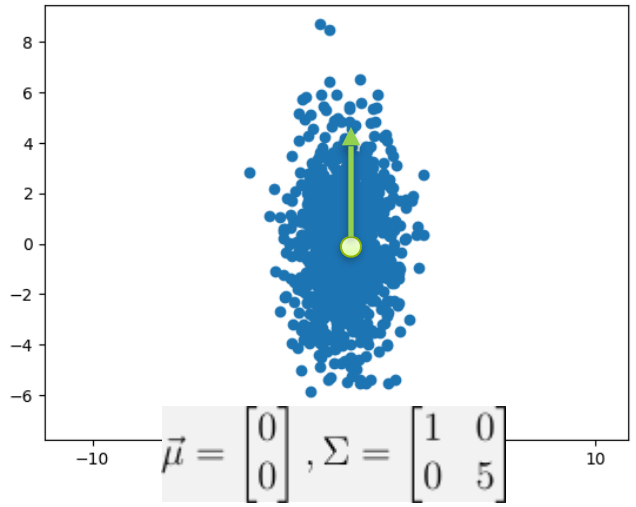
Welcome to the lecture!

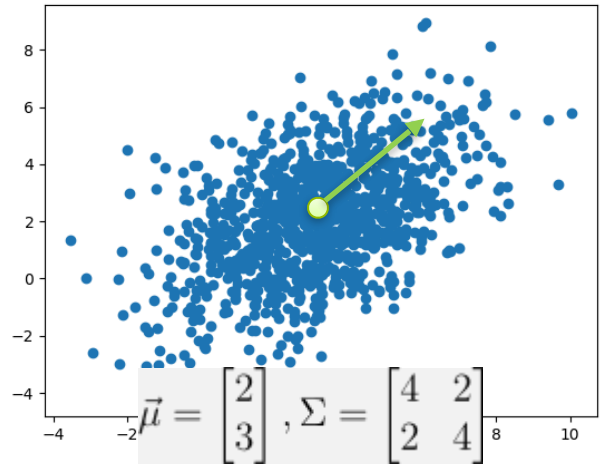
Prof. Dr.-Ing. Björn Giesler
Technische Hochschule Ingolstadt
AIMotion, AI and Sensor Data Fusion
Email: bjoern.giesler@thi.de

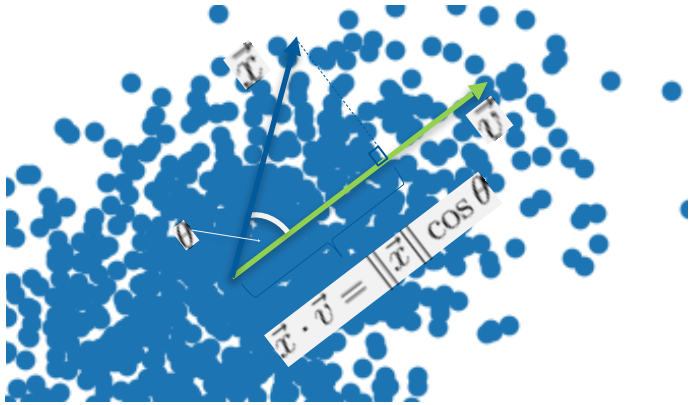
- **Preparing for the Exam**
  - Use the slides!!!
    I am updating them to have a bit more consistent variable naming etc., but everything in the exam is in the slides, and in the exercises.
  - Use the exercises!
    I will not have time unfortunately to create more exercise sheets.
    Maybe AI can prepare some sheets for you?
  - I will publish a list of literature, Youtube videos etc. that can also help you prepare.
  - Study together, in pairs or groups!
    There's always someone in a group who has good questions, or good understanding.
- These recap slides are *not enough* to prepare for the exam. Use the original slides and the exercises.

Co-Variance and Eigenvalues









- The covariance matrix ($\sum$ from now on) defines spread (variance) and orientation (covariance) of our data.
- If we want to represent the covariance matrix with a vector and its magnitude, we should try to find the vector that points into the direction of the largest spread of the data.
- The spread (variance) in this direction can give us the magnitude.
- Let‘s call that vector $\vec{v}$.
- We can project all of our data onto that vector with the dot product $\vec{v}^\top \vec{x} = \|\vec{x}\| \cos\theta$.
- The variance of the projected data is $\vec{v}^\top \Sigma \vec{v}$.
- We want to find the vector $\vec{v}$ that maximizes $\vec{v}^\top \Sigma \vec{v}$.

- **Maximizing any function of the form $\vec{v}^\top \Sigma \vec{v}$ with respect to $\vec{v}$ can be done with the so called *Rayleigh Quotient:***

$$R(M, \vec{x}) = \frac{\vec{x}^\top M \vec{x}}{\vec{x}^\top \vec{x}}$$

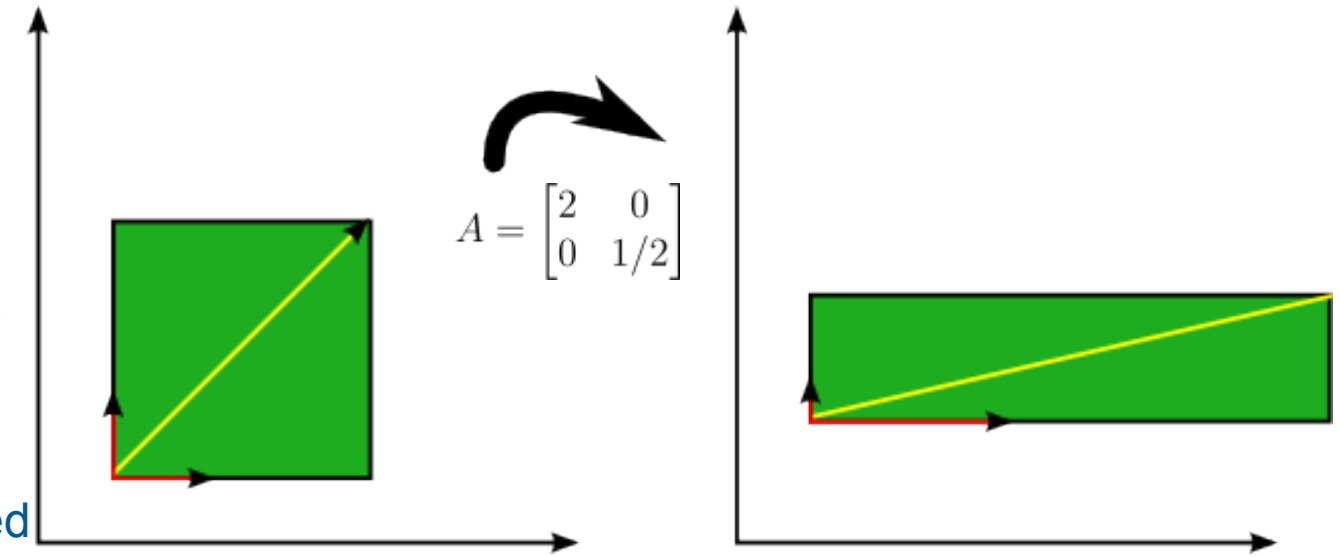
- **It can be shown that $R(M, \vec{x})$ reaches its maximum value $\lambda_{\max}$ when $\vec{x} = \vec{v}_{\max}$, the largest eigenvalue of M.**
- **The largest eigenvector of the covariance matrix always points in the direction of the largest variance,**
  and its magnitude equals the corresponding eigenvalue.
- **The second largest eigenvector is always orthogonal to the largest eigenvector, and points into the direction of the second largest spread of data.**
- ...and so on for all *n* dimensions of the matrix.

Revisiting Eigenvectors and Eigenvalues

- *Eigenvectors* / *eigenvalues* have many important applications in machine learning and computer vision.
- An eigenvector is a vector whose direction remains unchanged when a linear transformation is applied.
- Consider the scaling transform $\vec{v}' = A\vec{v}$ on the right. The yellow vector changes direction, the red vectors do not.
- Therefore the red vectors are called eigenvectors of A.
- Generally, the eigenvectors $\vec{v}_i$ of a matrix A are vectors that satisfy

$$A\vec{v}_i = \lambda_i \vec{v}_i$$

where λ_i are corresponding scalars called eigenvalues.



Revisiting Eigenvectors and Eigenvalues

- We can rewrite the eigenvector equation from the next slide as follows:

$$A\vec{v}_i = \lambda_i \vec{v}_i$$
$$A\vec{v}_i - \lambda_i \vec{v}_i = 0$$
$$\vec{v}_i(A - \lambda_i I) = 0$$

  where I is the identity matrix of the same dimensions as A.

- This is of course true for the trivial solution of $\vec{v}_i = \vec{0}$.
- If $\vec{v}_i$ is not the null vector, $\vec{v}_i(A - \lambda_i I) = 0$ can only be defined if $(A - \lambda_i I)$ is not invertible.
- For a square matrix not to be invertible, its determinant must be zero.
- Therefore, we can find λ_i by solving $\mathrm{Det}(A - \lambda I) = 0$.

Eigenvectors and Inverse Matrices

- Eigenvector of a matrix $\boldsymbol{A}$ corresponding to eigenvalue λ_i given by

$$(\boldsymbol{A} - \lambda_i \boldsymbol{I})\, \boldsymbol{v}_i = \boldsymbol{0} \quad \text{or} \quad \boldsymbol{A}\boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i$$

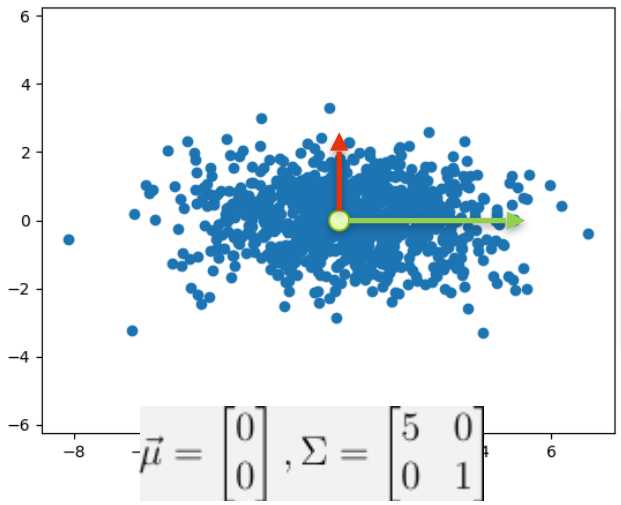
- Multiplying from the left with $\boldsymbol{A}^{-1}$ we obtain

$$\boldsymbol{v}_i = \lambda_i \boldsymbol{A}^{-1} \boldsymbol{v}_i$$

$$\leftrightarrow \boldsymbol{A}^{-1} \boldsymbol{v}_i = \frac{1}{\lambda_i} \boldsymbol{v}_i$$

- So, the eigenvector $\boldsymbol{v}_i$ of $\boldsymbol{A}$ is also an eigenvector of $\boldsymbol{A}^{-1}$ for the eigenvalue $1/\lambda_i$

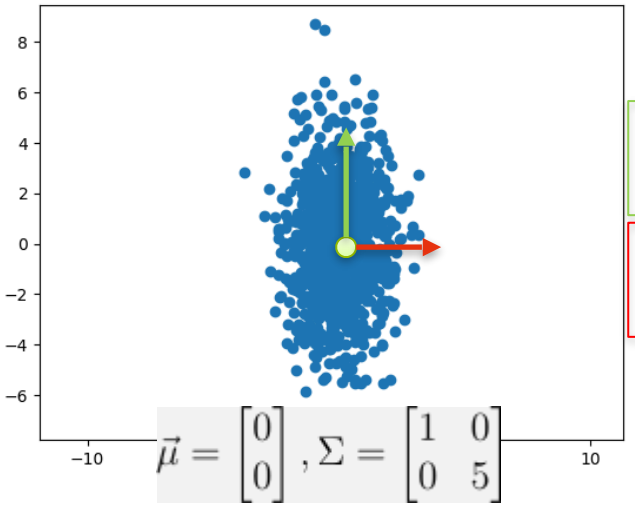
Some examples:



$$\vec{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\|v_1\| = \lambda_1 = 5, v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

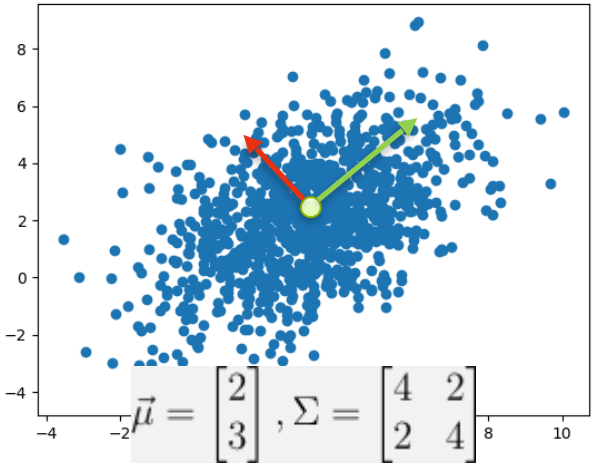
$$\|v_2\| = \lambda_2 = 1, v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$



$$\vec{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 5 \end{bmatrix}$$

$$\|v_1\| = \lambda_1 = 5, v_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\|v_2\| = \lambda_2 = 1, v_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

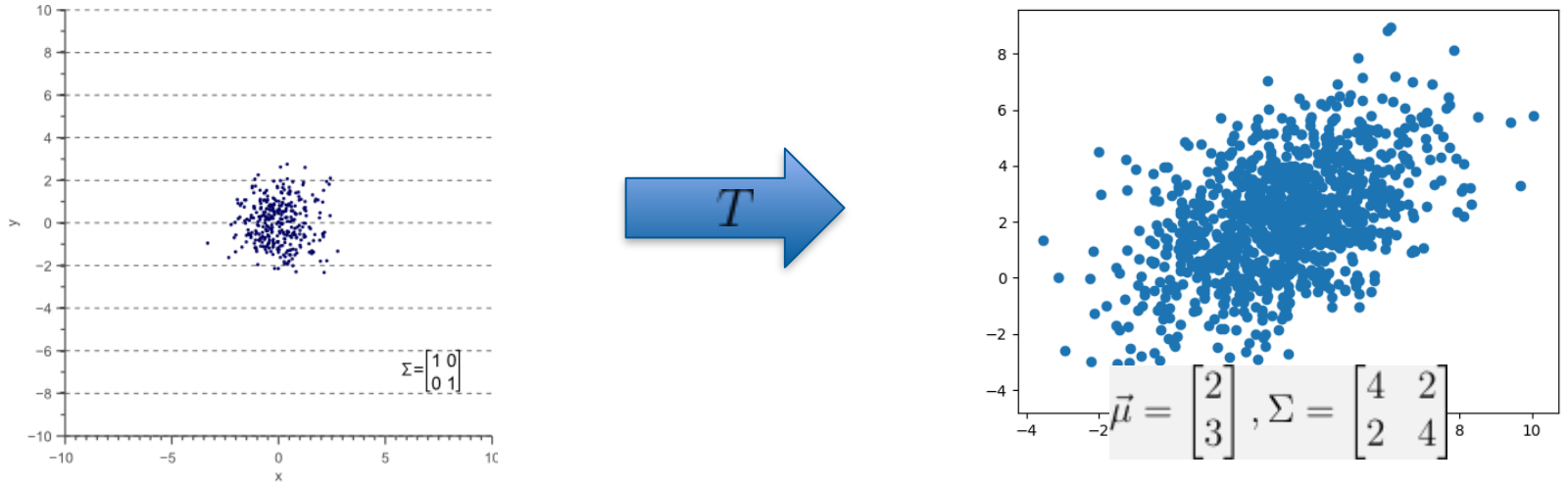


$$\vec{\mu} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \Sigma = \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix}$$

$$\|v_1\| = \lambda_1 = 6, v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\|v_2\| = \lambda_2 = 2, v_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Covariance Matrix as a Linear Transformation



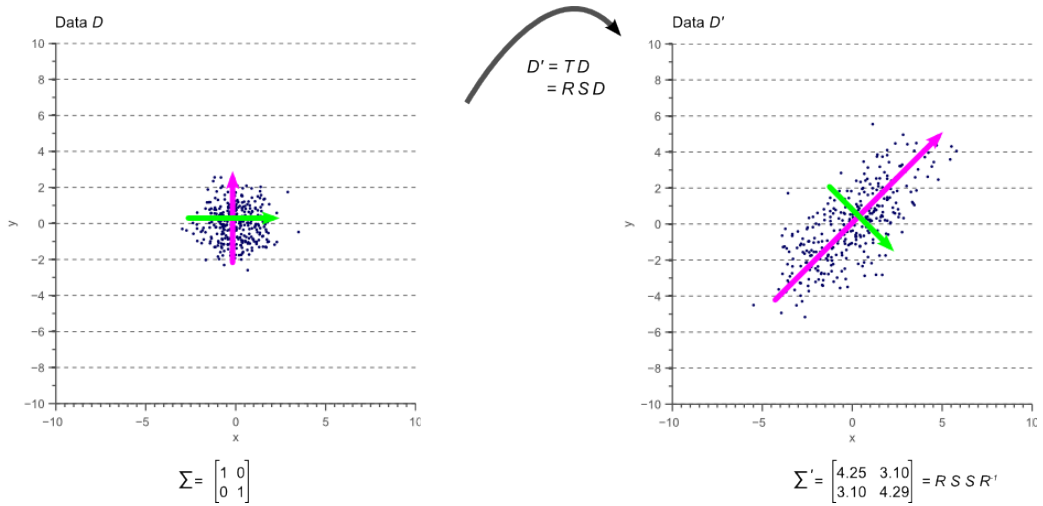
D $\quad D' = TD + \vec{\mu}$ $\quad D'$

where T is the product of a rotation matrix R and a scaling matrix S:

$$T = RS$$

$\vec{\mu}$ is a translation vector representing the mean.

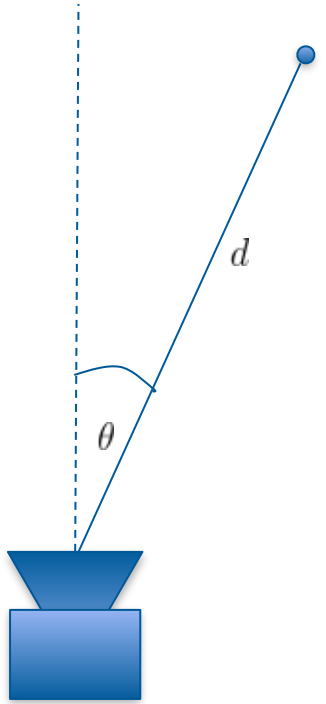
- **If we have**
  - **an original data set D with a covariance matrix Σ, and**
  - **a linear transformation T given by a rotation R, translation (mean shift) $\vec{\mu}$, and scaling matrix S, such that $D' = TD + \vec{\mu} = RSD + \vec{\mu}$,**
- **the covariance matrix of the transformed data set D' is calculated by $\Sigma' = TT^{\top}\Sigma = RS(SR)^{\top}\Sigma$.**



Putting it all together – Sensor Modelling Example

- **Let‘s assume we have a sensor that delivers distance over azimuth.**
- **We take 1000 measurements at various angles and distances and find:**
  - all measurements within $\pm 3^\circ$ across the whole field of view
  - all measurements within $\pm 1\%$ depending on distance.

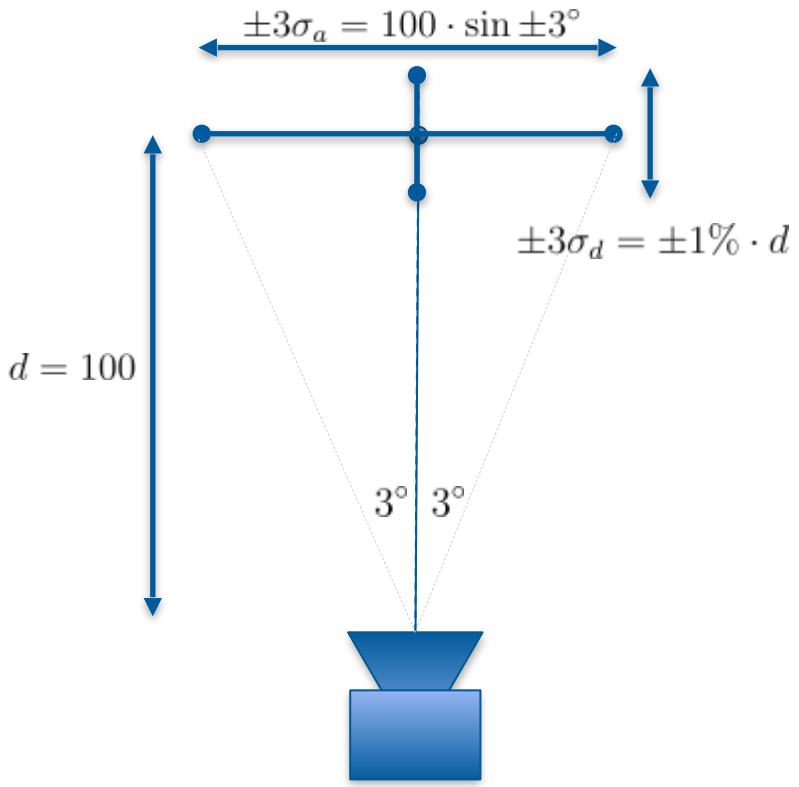
- **What is the covariance matrix of an object measured at 20°of azimuth, 100m of distance?**



- **We start with an unrotated normal distribution of** $\Sigma_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$.
- **Since we empirically measured all measurements within** $\pm 1\%$ **of distance, we can assume** $3\sigma_d = 100 \cdot 1\%$

  or $\sigma_d = \frac{1}{3}$ **meters.**
- **Since we empirically measured all measurements with** $\pm 3^\circ$ **of azimuth, we can assume** $3\sigma_a = 100 \cdot \sin 3^\circ$

  or $\sigma_a \approx \frac{5}{3}$ **meters.**
- **So our unrotated data is scaled by** $\begin{bmatrix} \frac{5}{3} & 0 \\ 0 & \frac{1}{3} \end{bmatrix}$.



- **We see the object at 20° of azimuth, so we need to rotate by**

$$\begin{bmatrix} \cos 20° & -\sin 20° \\ \sin 20° & \cos 20° \end{bmatrix} \approx \begin{bmatrix} 0.94 & -0.342 \\ 0.342 & 0.94 \end{bmatrix}$$

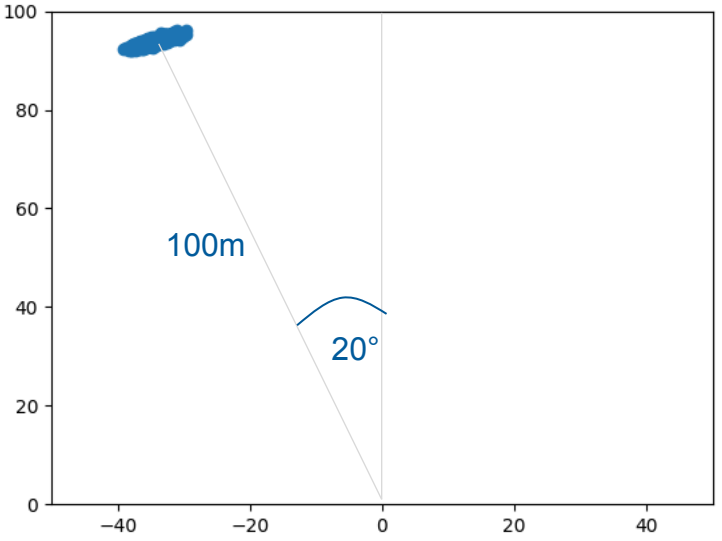
- **For a data transform of** $T = RS = \begin{bmatrix} \cos 20° & -\sin 20° \\ \sin 20° & \cos 20° \end{bmatrix} \begin{bmatrix} \frac{5}{3} & 0 \\ 0 & \frac{1}{3} \end{bmatrix}$

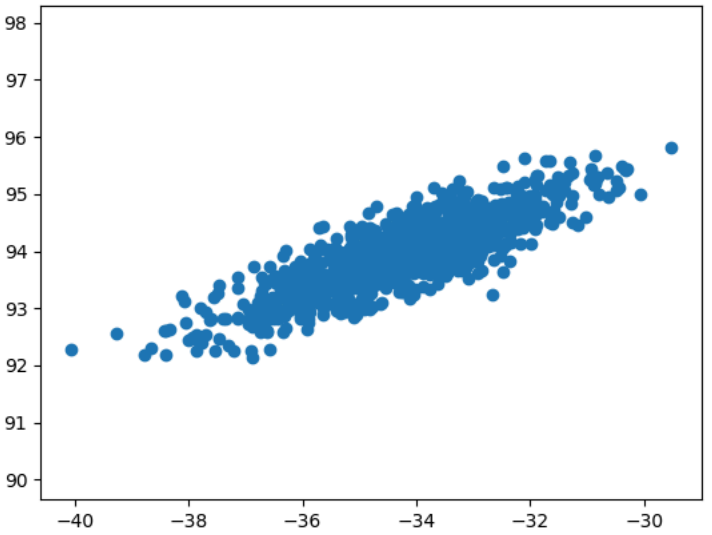
we obtain a covariance matrix of

$$\Sigma' = RS(SR)^T = \begin{bmatrix} \cos 20° & -\sin 20° \\ \sin 20° & \cos 20° \end{bmatrix} \begin{bmatrix} \frac{5}{3} & 0 \\ 0 & \frac{1}{3} \end{bmatrix} \begin{bmatrix} \frac{5}{3} & 0 \\ 0 & \frac{1}{3} \end{bmatrix} \begin{bmatrix} \cos 20° & \sin 20° \\ -\sin 20° & \cos 20° \end{bmatrix}$$

and can use this to generate a scatter plot of the multivariate normal distribution at

$$\vec{\mu} = \begin{bmatrix} 100 \sin \theta \\ 100 \cos \theta \end{bmatrix} \approx \begin{bmatrix} 34.2 \\ 93.97 \end{bmatrix}.$$





Kalman Filter

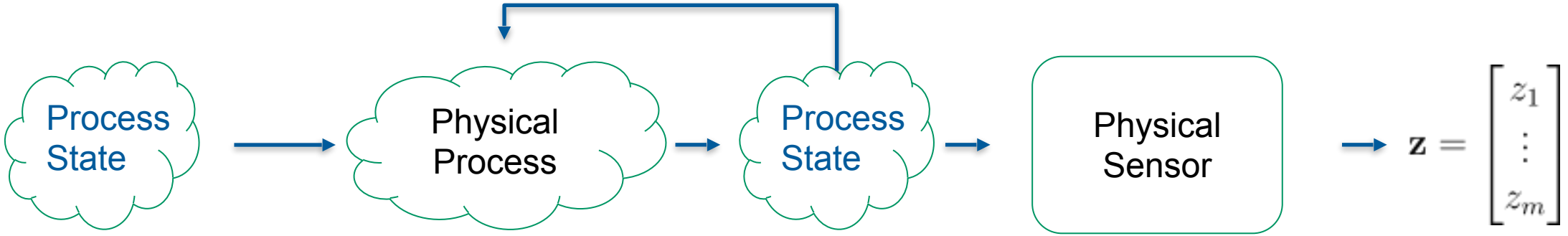
Interrelating Kalman, Extended Kalman, and Unscented Kalman Filters

	Input Error Distribution	Function Transform	Output Error Distribution
Kalman Filter	Approximate	Exact (linear)	Approximate
Extended Kalman	Approximate	Approximate	Approximate
Unscented Kalman	Approximate	Exact (nonlinear)	Approximate

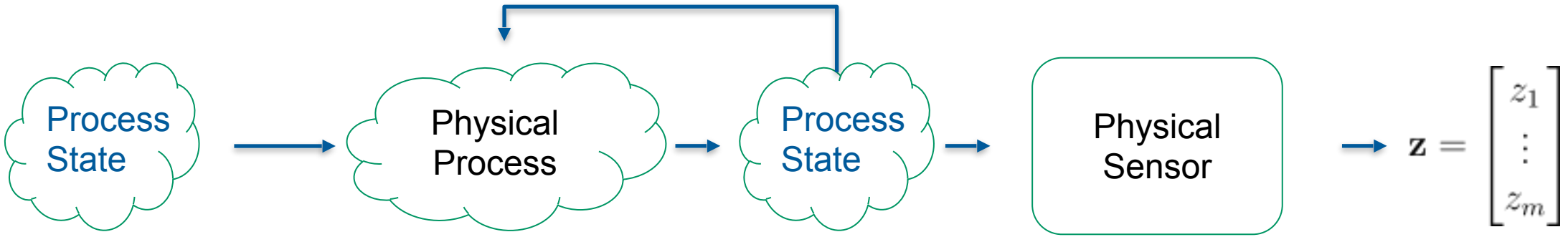
UKF Advantage:

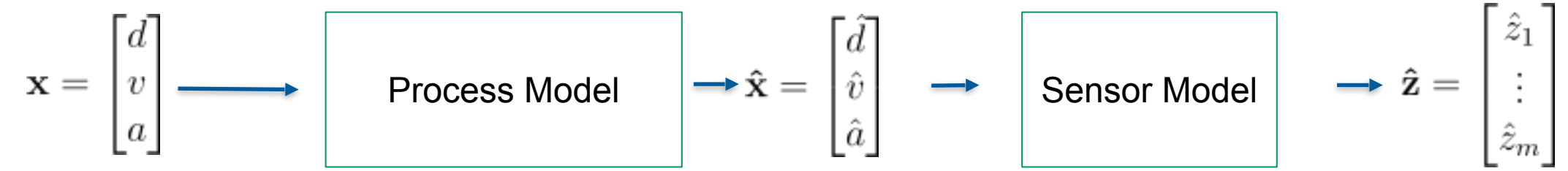
1. Nonlinear motion and sensor models do not need to be known.
2. No need to compute Jacobians.
3. Motion and sensor models need not be continuous or differentiable.
4. Computational complexity is similar to EKF.
5. Unscented Transform is accurate up to the 3rd moment for Gaussians.

State, Process, Measurement



Modeling State, Process, Measurement

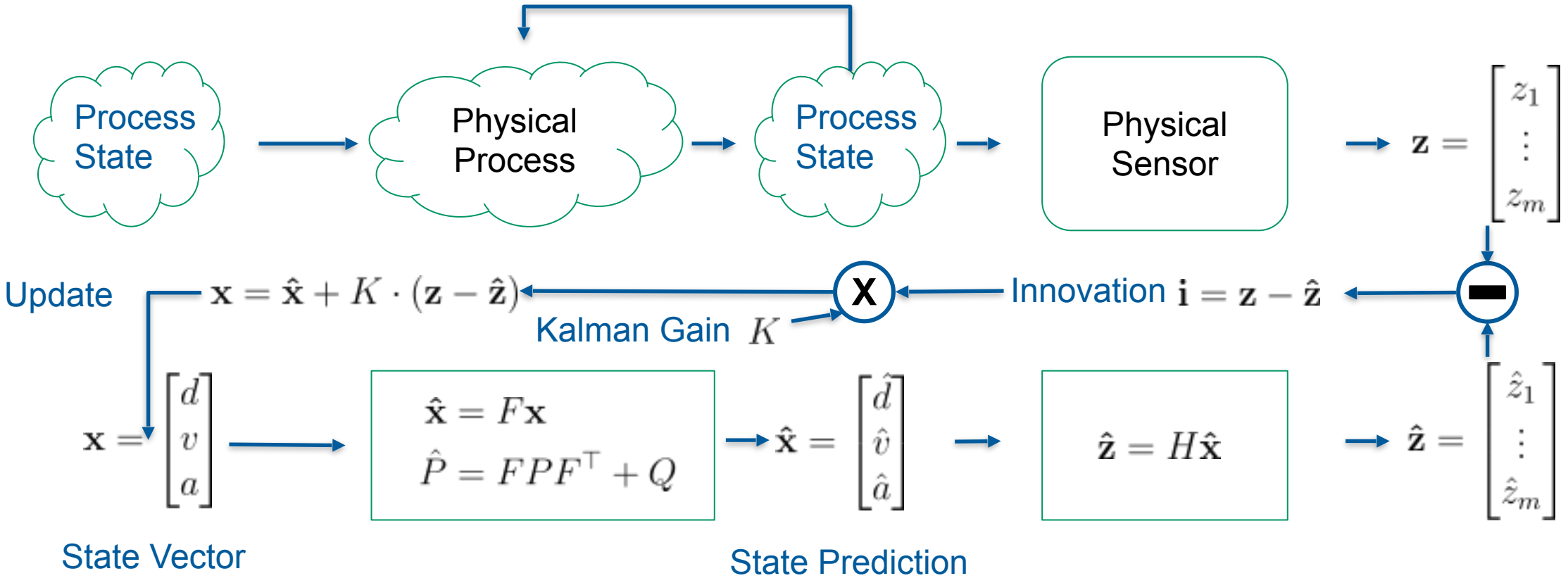


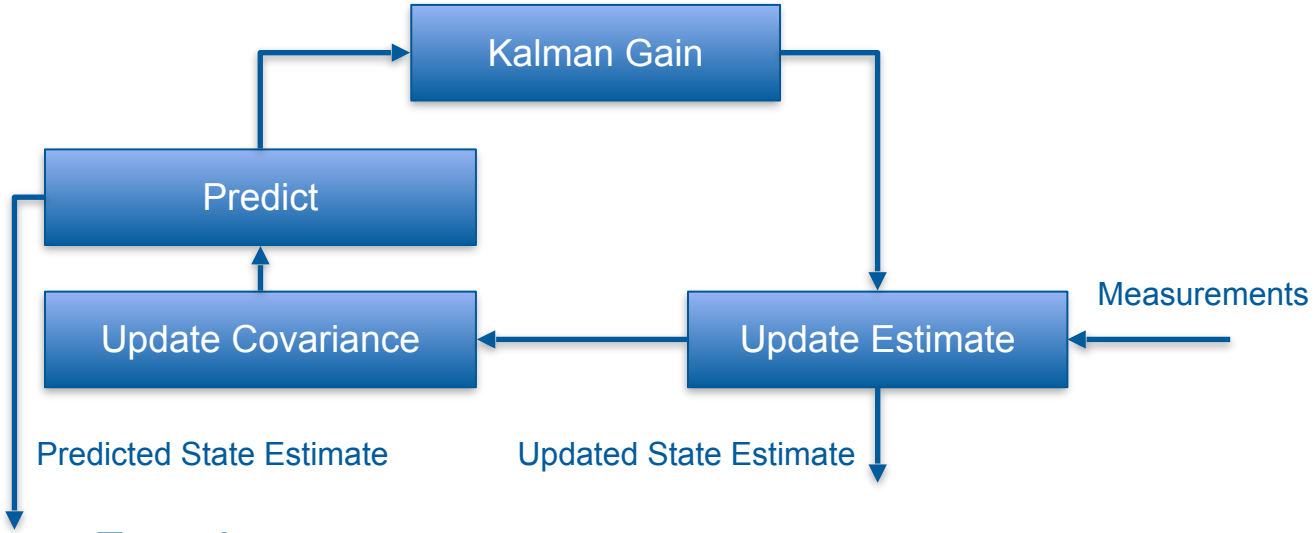


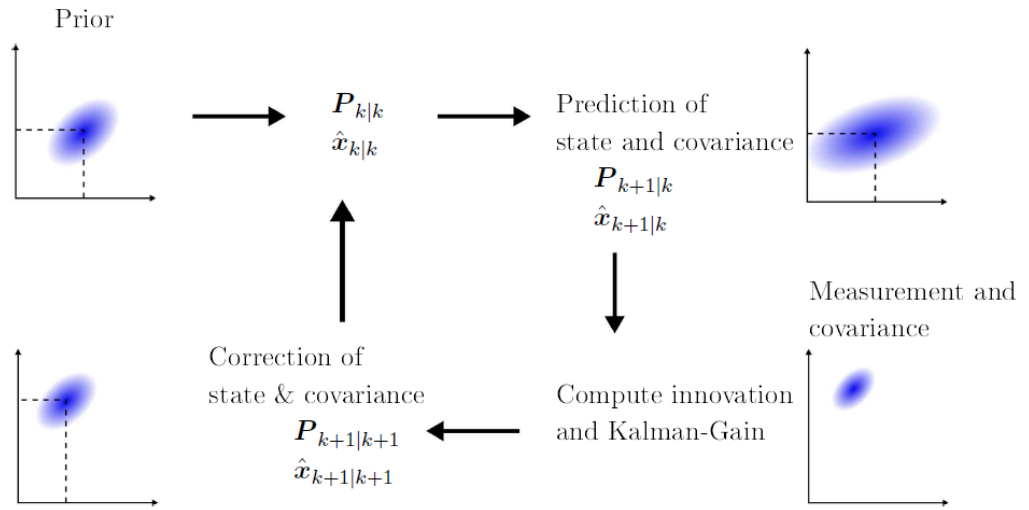
State Vector

State Prediction

Kalman Filter for Linear Processes







Iterations of the Kalman Filter

Equations:

- Predict

$$\hat{\mathbf{x}}_{t+1|t} = A_t \hat{\mathbf{x}}_{t|t} \quad (+B_t \mathbf{u}_t)$$

if you are adding some input to the system instead of just observing

$$P_{t+1|t} = A_t P_{t|t} A_t^\top + Q_t$$

- Kalman Gain

$$K_{t+1} = P_{t+1|t} H_{t+1}^\top (H_{t+1} P_{t+1|t} H_{t+1}^\top + R_{t+1})^{-1}$$

- Update Estimate

$$\hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_{t+1}(\mathbf{z}_{t+1} - H\hat{\mathbf{x}}_{t+1|t})$$

- Update Covariance

$$P_{t+1|t+1} = (I - K_{t+1} H_{t+1}) P_{t+1|t}$$

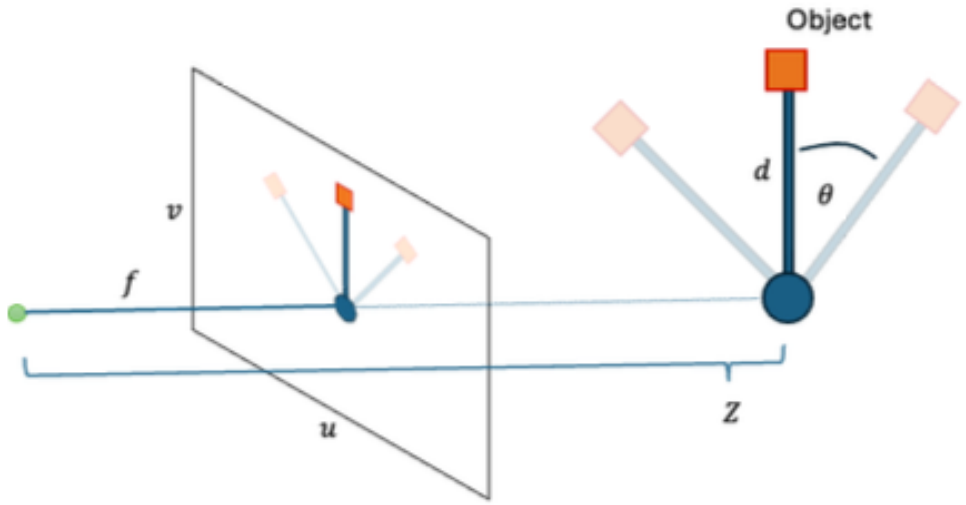
Role of the Kalman Gain:
Project innovation $\mathbf{z} - H\hat{\mathbf{x}}$ from measurement into state space
Interpolate between old state estimate and projected innovation

Extended Kalman Filter - Equations

- **For the prediction, we need the Jacobian of the state transition function:** $F_t = \frac{\partial f}{\partial x}(\hat{\mathbf{x}}_{t|t}, \mathbf{w}_{t|t})$
- **With this, we can formulate the prediction equations:**
  - Predicted state estimate: $\hat{\mathbf{x}}_{t+1|t} = f(\hat{\mathbf{x}}_{t|t}, \mathbf{w}_{t+1|t})$
  - Predicted covariance estimate: $P_{t+1|t} = F_t P_{t|t} F_t^\top + Q_t$

- **For the update, we need the Jacobian of the measurement function:** $H_t = \frac{\partial h}{\partial x}(\hat{\mathbf{x}}_{t+1|t}) = \frac{\partial h}{\partial x}(f(\hat{\mathbf{x}}_{t|t}, \mathbf{w}_t))\,.$
- **With this, we can formulate the update equations:**
  - Kalman Gain: $K_{t+1} = P_{t+1|t} H_{t+1}^\top (H_{t+1} P_{t+1|t} H_{t+1}^\top + R_{t+1})^{-1}$ (unchanged from linear KF)
  - State Update: $\hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_{t+1}(\mathbf{z}_{t+1} - h(\hat{\mathbf{x}}_{t+1|t}))$
  - State Covariance Update: $P_{t+1|t+1} = (I - K_{t+1} H_{t+1}) P_{t+1|t}$ (unchanged from linear KF)

EKF Example



- Tracking an object moved by a robot arm at distance Z, seen by a camera with focal distance f.
- Parameters: length of arm d, rotational angle θ_t, varying with rotational velocity ω_t.
- State vector: $\mathbf{x}_t = (d, \theta_t, \omega_t)^\top$.
- State transition function: $\mathbf{x}_{t+1} = g(\mathbf{x}_t) = \begin{bmatrix} d \\ \theta_t + \omega_t \cdot \Delta t \\ \omega_t \end{bmatrix}$
- Measurement function: $\mathbf{z}_t = h(\mathbf{x}_t) = \frac{f}{Z} \begin{bmatrix} d \cos \theta_t \\ d \sin \theta_t \end{bmatrix}$.

- **Jacobian of transition function:**

$$J_g = \begin{bmatrix} \frac{\partial d}{\partial d} & \frac{\partial d}{\partial \theta_t} & \frac{\partial d}{\partial \omega_t} \\ \frac{\partial \theta_t + \omega_t \cdot \Delta t}{\partial d} & \frac{\partial \theta_t + \omega_t \cdot \Delta t}{\partial \theta_t} & \frac{\partial \theta_t + \omega_t \cdot \Delta t}{\partial \omega_t} \\ \frac{\partial \omega_t}{\partial d} & \frac{\partial \omega_t}{\partial \theta_t} & \frac{\partial \omega_t}{\partial \omega_t} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}.$$

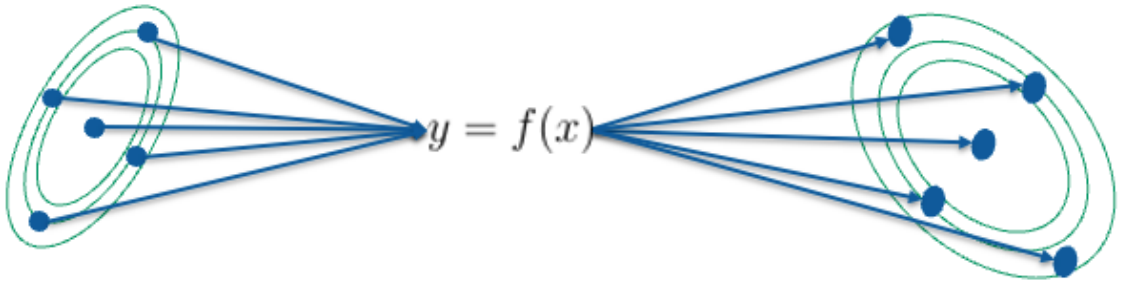
- **Jacobian of measurement function:**

$$J_h = \frac{f}{Z} \begin{bmatrix} \frac{\partial d \cos\theta_t}{\partial d} & \frac{\partial d \cos\theta_t}{\partial \theta_t} & \frac{\partial d \cos\theta_t}{\partial \omega_t} \\ \frac{\partial d \sin\theta_t}{\partial d} & \frac{\partial d \sin\theta_t}{\partial \theta_t} & \frac{\partial d \sin\theta_t}{\partial \omega_t} \end{bmatrix} = \frac{f}{Z} \begin{bmatrix} \cos\theta_t & -d\sin\theta_t & 0 \\ \sin\theta_t & d\cos\theta_t & 0 \end{bmatrix}.$$

- **Note that without info on f and Z, we can't really compute this filter; but we can make assumptions and treat $a = \frac{f}{Z}$ as a scaling factor; with that, we can still filter for θ, ω even though d will be out of scale.**

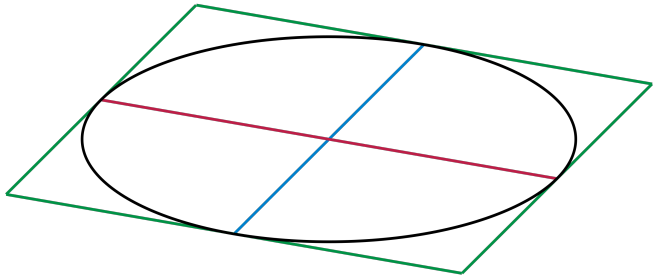
Unscented Kalman Filter

- **As we have seen, the Extended Kalman Filter helps us to get rid of *some* nonlinearities, provided they are reasonably well-behaved.**
- **It deteriorates for more complex functions.**
- **Enter the Unscented Kalman Filter. Basic idea:**
  - Sample the error distribution at *precisely chosen* points surrounding the mean
  - Transform these points through the state transition function
  - Use the transformed points to approximate the output distribution.

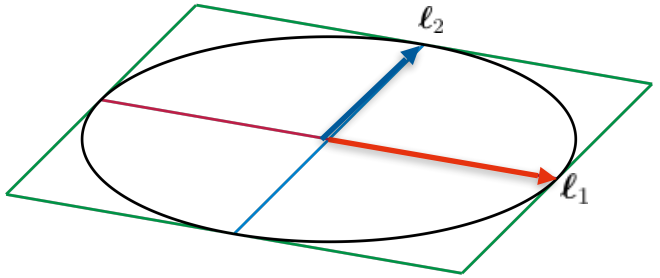


The Unscented Transform – Choosing Sigma Points

- **The Cholesky Decomposition is equivalent to a particular choice of conjugate axes of an ellipsoid.**
  - For an ellipsoid, two diameters are conjugate if and only if the tangent line to the ellipse at an endpoint of one diameter is parallel to the other diameter.
  - This is not the same as reconstructing the eigenvectors, but it gives symmetry axes that we can sample equidistant points on.

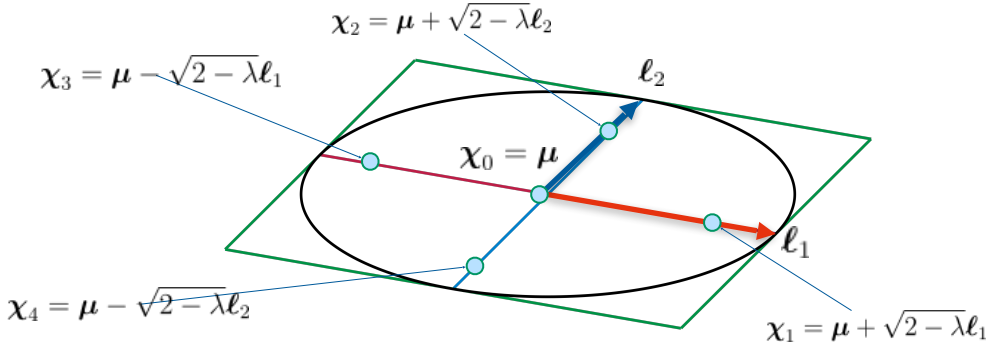


Two conjugate axes (red, blue) of an ellipsis $\sum$ ...



...obtained as column vectors $\boldsymbol{\ell}_i$ of the Cholesky Decomposition

$$\Sigma = LL^\top ...$$



...give the positions of the sigma points.

The Unscented Transform – Choosing Sigma Points

- We generate $2n + 1$ sigma points

$$\boldsymbol{\chi}_0 = \boldsymbol{\mu},$$

$$\boldsymbol{\chi}_i = \begin{cases} \boldsymbol{\mu} + \sqrt{n+\lambda} \cdot \boldsymbol{\ell}_i & \text{if } i = 1, \ldots, n \\ \boldsymbol{\mu} - \sqrt{n+\lambda} \cdot \boldsymbol{\ell}_{i-n} & \text{if } i = n+1, \ldots, 2n \end{cases}$$

where ℓ_i is the i-th column of the Cholesky decomposition matrix $L \in R^{n \times n}$

$$\boldsymbol{L} \cdot \boldsymbol{L}^{\mathrm{T}} = \boldsymbol{P}_{k|k}$$

and λ is a chooseable parameter.

- Every sigma point $\boldsymbol{\chi}_i$ gets assigned a corresponding weight w_i with

$$w_0 = \frac{\lambda}{n+\lambda}$$

$$w_i = \frac{1}{2(n+\lambda)}, \quad i = 1 \ldots n$$

The weights are reconstructed from assumptions about the mean and variance ==> check slides!

Unscented Kalman-Filter (UKF) – Prediction Step

1. Compute sigma-points $\chi_{i,t|t}$ based on the current estimate $\hat{x}_{t|t}$ and covariance $P_{t|t}$.
2. Predict: Predict the sigma-points employing the nonlinear dynamics, i.e.

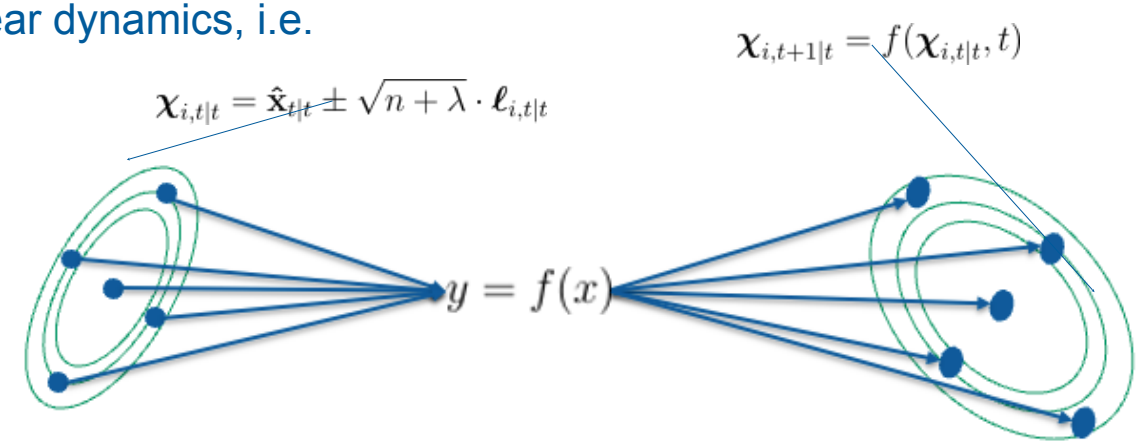
$$\boldsymbol{\chi}_{i,t+1|t} = f(\boldsymbol{\chi}_{i,t|t}, t)$$

3. Approximate the predicted mean

$$\hat{\mathbf{x}}_{t1+1|t} = \bar{\boldsymbol{\chi}} = \sum_{i=0}^{2n} w_i \boldsymbol{\chi}_{i,t+1|t}$$

and covariance:

$$P_{t+1|t} = Q_t + \sum_{i=0}^{2n} w_i (\boldsymbol{\chi}_{i,t+1|t} - \bar{\boldsymbol{\chi}})(\boldsymbol{\chi}_{i,t+1|t} - \bar{\boldsymbol{\chi}})^\top.$$



Unscented Kalman-Filter (UKF) – Measurement Update

1. Computing/Using sigma- points $\chi_{i,t+1|t}$ based on the predicted estimate $\hat{x}_{t+1|t}$ and covariance $P_{t+1|t}$
2. Compute a predicted measurement from the weighted sum of predictions for each sigma point:

$$\hat{\mathbf{z}}_{t+1|t} = \sum_{i=0}^{2n} w_i \mathbf{z}_{i,t+1|t} \qquad \text{with} \qquad \mathbf{z}_{i,t+1|t} = h(\boldsymbol{\chi}_{i,t+1|t}, t)$$

3. Approximate the measurement covariance and the cross-covariance between state and measurement:

$$P_{zz} = \sum_{i=0}^{2n} w_i (\mathbf{z}_{i,t+1|t} - \hat{\mathbf{z}}_{t+1|t})(\mathbf{z}_{i,t+1|t} - \hat{\mathbf{z}}_{t+1|t})^\top$$

$$P_{xz} = \sum_{i=0}^{2n} w_i (\boldsymbol{\chi}_{i,t+1|t} - \bar{\boldsymbol{\chi}})(\mathbf{z}_{i,t+1|t} - \hat{\mathbf{z}}_{t+1|t})^\top$$

4. Compute the Kalman-Gain and perform correction of state and covariance.

$$K_t = P_{xz} \cdot (R + P_{zz})^{-1} \qquad \hat{\mathbf{x}}_{t+1|t+1} = \hat{\mathbf{x}}_{t+1|t} + K_t \cdot (\mathbf{z}_{t+1} - \hat{\mathbf{z}}_{t+1|t})$$

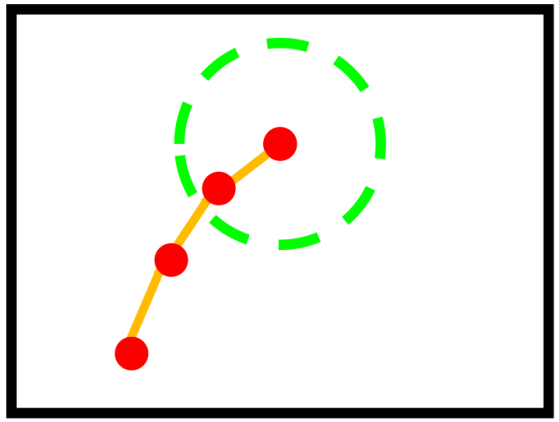
$$P_{t+1|t+1} = P_{t+1|t} - K_t P_{zz} K_t^\top.$$

Compare Kalman Filter and UKF Equations

	Linear	UKF
• Kalman Gain	$K_t = P_{t\|t-1} H^\top (H P_{t\|t-1} H^\top + R)^{-1}$	$K_t = P_{xz} \cdot (P_{zz} + R)^{-1}$
• Update Estimate	$\hat{\mathbf{x}}_{t+1\|t+1} = \mathbf{x}_{t+1\|t} + K_t \cdot (\mathbf{z}_{t+1} - H\hat{\mathbf{x}}_{t+1\|t})$	$\hat{\mathbf{x}}_{t+1\|t+} = \hat{\mathbf{x}}_{t+1\|t} + K_t \cdot (\mathbf{z}_{t+1} - \hat{\mathbf{z}}_{t+1\|t})$
• Update Covariance	$P_{t+1\|t+1} = P_{t+1\|t} - K_t H P_{t+1\|t}$	$P_{t+1\|t+1} = P_{t+1\|t} - K_t P_{zz} K_t^\top.$
• Predict	$\hat{\mathbf{x}}_{t+1\|t} = A\hat{\mathbf{x}}_{t\|t}$ $P_{t+1\|t} = A P_{t\|t} A^\top + Q$	$\hat{\mathbf{x}}_{t1+1\|t} = \bar{\chi}_{2n} = \sum_{i=0}^{2n} w_i \boldsymbol{\chi}_{i,t+1\|t}$ $P_{t+1\|t} = \sum_{i=0} w_i (\boldsymbol{\chi}_{i,t+1\|t} - \bar{\boldsymbol{\chi}})(\boldsymbol{\chi}_{i,t+1\|t} - \bar{\boldsymbol{\chi}})^\top + Q$

Multi Object Tracking

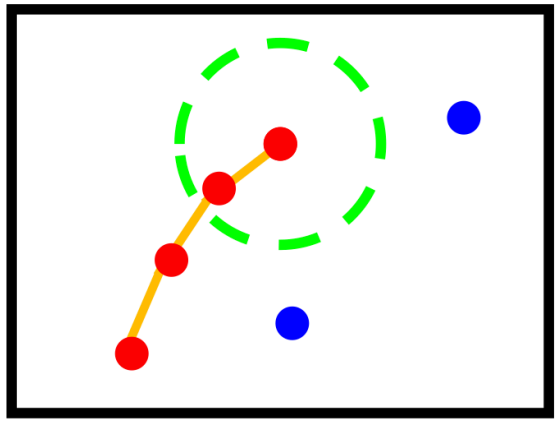
Motion Correspondence



1. Predictions may not be supported by measurements.

   Have the objects ceased to exist?
   ➔ remove them

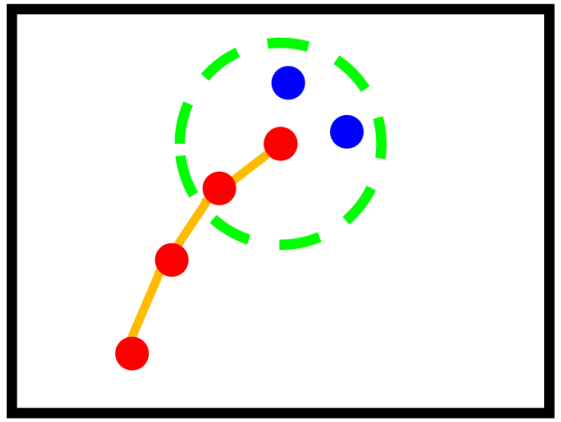
   Are they occluded?
   ➔ keep them



2. There may be unexpected measurements.

   Newly visible objects?
   ➔ set up tracks

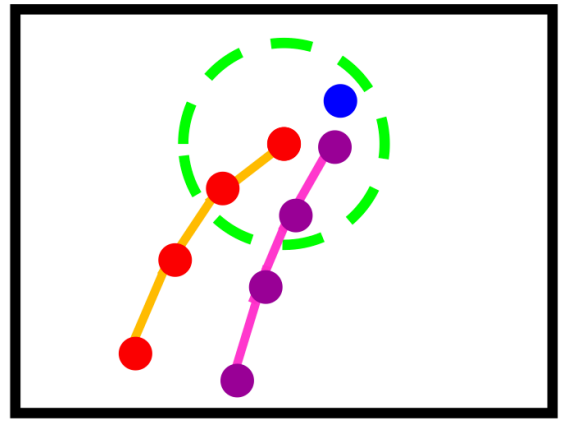
   Just noise?
   ➔ ignore them



3. More than one measurement may match a prediction.

   Which measurement is the correct one?

   What about the others?



4. A measurement may match to multiple predictions.

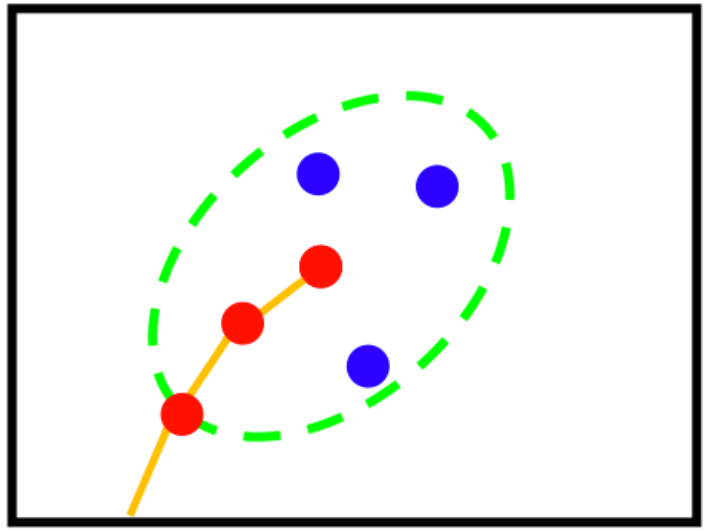
   Which track should the measurement be assigned to?

   What about the other track(s)?

Multi-Object Tracking Problem

- We represent a track by a state vector, e.g. $\mathbf{x} = \left[x, y, v_x, v_y\right]^\top$
- As the track evolves, we denote its state by the time index t: $\mathbf{x}^{(t)} = [x^{(t)}, y^{(t)}, v_x^{(t)}, v_y^{(t)}]^\top$
- At each time step, we get a set of observations / measurements: $\mathbf{Y}^{(t)} = \{\mathbf{y}_1^{(t)}, \ldots, \mathbf{y}_{M_t}^{(t)}\}$
- We now need to make the data association between tracks $\{\mathbf{x}_1^{(t)}, \ldots, \mathbf{x}_{N_t}^{(t)}\}$ and measurements $\{\mathbf{y}_1^{(t)}, \ldots, \mathbf{y}_{M_t}^{(t)}\}$:
- Association $z_l^{(t)} = j \text{ iff } \mathbf{y}_j^{(t)}$ is associated with $\mathbf{x}_l^{(t)}$.

The Mahalanobis Distance as a measure for track similarity



- Our Kalman Filter state of track $\mathbf{x}_l$ is given by the prediction and covariance $\hat{\mathbf{x}}_l^{(t)}, P_l'(t).$
- We have defined the *innovation* that measurement $\mathbf{z}_j$ brings to track $\mathbf{x}_l$ at time t as $\mathbf{i}_{j,l}^{(t)} = (\mathbf{z}_j^{(t)} - \hat{\mathbf{x}}_l^{(t)}).$
- With this, we can write the observation likelihood shortly as $p(\mathbf{z}_j^{(t)}|\mathbf{x}_l^{(t)} \sim \exp\{-\frac{1}{2}\mathbf{i}_{j,l}^{(t)\top} P_l'^{-1}\mathbf{i}_{j,l}^{\top}\}.$

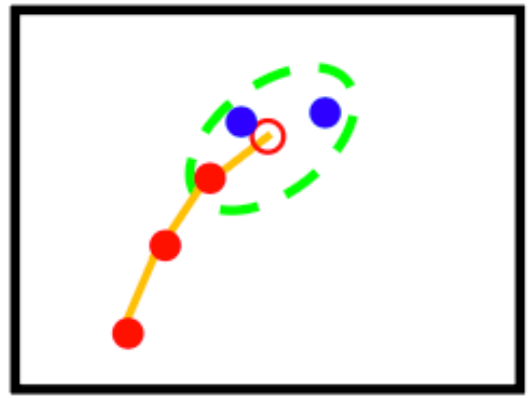
Mahalanobis Distance

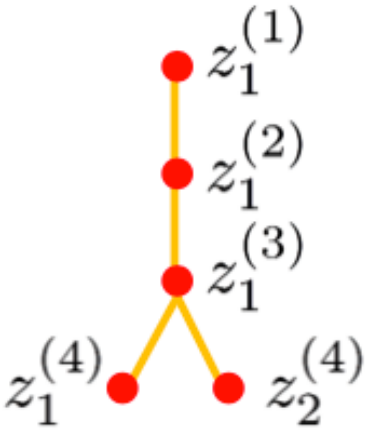
- We can define the ellipsoidal gating or validation volume as $V^{(t)}(\gamma) = \{\mathbf{z} : (\mathbf{y} - \hat{\mathbf{x}}_l^{(t)})^\top P_l'^{(t)-1}(\mathbf{y} - \hat{\mathbf{x}}_l^{(t)}) \leq \gamma\}.$
- This is equivalent to saying "all $\mathbf{y}$ with a Mahalanobis distance $\leq \gamma$".

$$z_l^{(t)} = \underset{j}{\operatorname{argmin}} \ (\hat{\mathbf{x}}_l^{(t)} - \mathbf{y}_j^{(t)})^\top P_l'^{-1}(\hat{\mathbf{x}}_l^{(t)} - \mathbf{y}_j^{(t)})$$

Track Splitting Filter

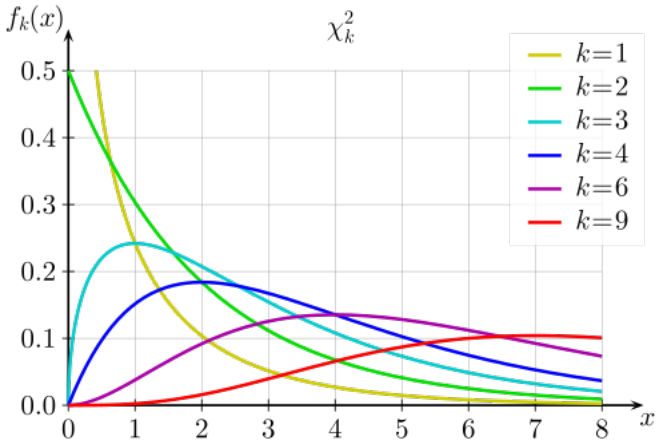
- **The Nearest-Neighbor filter still has uncertainties and options that can't really be resolved**
  - Uncertain decision making
  - ...through gating decision making functions
- **Rather than arbitrarily assigning the "closest" (by whatever norm) measurement, form a tree.**
- **Branches denote alternate assignments. No *decision* is made at this stage; decisions are postponed until additional measurements have been gathered.**
- **Potential problem:**
  Track trees can quickly become very large due to combinatorial explosion.
- **We need some measure of the likelihood so we can prune the tree.**





Track Splitting Filter – Pruning Strategies

- **In order to keep this feasible, need to apply pruning.**
- **Deleting unlikely tracks**
  - May be accomplished by comparing $\lambda(t)$, which follows a χ^2 distribution with $t \cdot n_z$ degrees of freedom, with a threshold α (set according to χ^2tables).
  - In long tracks, $\lambda(t)$ gets dominated by old terms and responds very slowly to new ones.
  - ➔ Use sliding window to limit track length, or implement exponential decay.
- **Merging track nodes**
  - If the state estimates of two track nodes are similar, merge them.
  - E.g. if both tracks validate identical subsequent measurments.
- **Rank tracks based on $\lambda(t)$, and only keep the most likely N tracks.**



χ^2 distribution for different degrees of freedom



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!